Computational Results for a Quantum Computing Application in Real-Life Finance

W. Bernard Lee
HedgeSPA Pte. Ltd.
1 Kay Siang Road #12-02
Singapore, 248922
bernard.lee@hedgespa.com

Anthony G. Constantinides
Financial Signals Processing Lab.
Imperial College London
London, UK SW7 2AZ
a.constantinides@imperial.ac.uk

2023 IEEE International Conference on Quantum Computing and Engineering (QCE) | 979-8-3503-4323-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/QCE57702.2023.00054

***Abstract*—This article presents the computational results obtained from a real-life implementation of quantum computing application in investment analytics, which algorithm was described by a previously published paper [5]. The new results reported in this article include: 1) speed-up tests based on idealized problems; 2) speed-ups computed on data from a professional platform, accounting for imperfections on very large financial datasets in finance; and 3) speed-ups from using similar data to solve analogous problems such as implied graphs to explain factor cuts. Furthermore, the article addresses the implementation challenges and computational results, in terms of both speed-ups and error rates, along with any other alternative hardware solutions (if available) such as GPUs for certain users who prefer classical computer solutions. That way, users can compare and contrast the state of the art today and in the foreseeable future. Overall, this article provides insights into the potential of quantum computing to revolutionize the field of finance and highlights the progress made in leveraging artificial intelligence technology for solving real-world problems.**

***Index Terms*—Graph Neural Network, Artificial Intelligence, Quantum Computing, Computational Speed-ups, Investment Analytics, Very Large Financial Datasets**

I. A Quantum/Graph Application in Finance

A. Background Context

HedgeSPA is a boutique investment analytics firm with a relatively small team of global professionals, funded by the Singapore Prime Minister's Office, and founded by a former Managing Director of the Portfolio Management Group of BlackRock in New York. Our core technology is an AI engine, combined with smart human-centric interfaces and a large-scale real-time data infrastructure. The company applies this technology to areas where we have deep domain expertise, such as fintech and sustainability.

Our AI technology is based on Graph Neural Network (GNN), an emerging new technology used by the likes of NASA for advanced control applications. The company's lineage can be traced to the founder's Ph.D. advisor, Professor Nicos Constantinides, who is commonly respected as the father of graph theory. We are working with his good friend Professor Tony Constantinides, a professor of electrical engineering pioneering this field.

HedgeSPA has earned global recognition with its numerous industry awards and press mentions. It has a considerable list of R&D publications at top scientific journals and collaborates with top-tier scientific research institutions. Our track record speaks for itself. We have achieved as much as double-digit outperformance without leverage or country and sector biases (as shown in Graph 1).

To elaborate further, our fintech platform uses an AI-Bot interface reading a large number of institutional-quality investment analytics reports generated on customer portfolios automatically (as shown in Graph 2), resulting in scientifically validated double-digit outperformance across 5 different global markets (as shown in Graph 3). An institutional client can deploy this solution as a multi-tier core investment platform architecture that can meet the stringent partnership technology standards of Amazon Web Services (as shown in Graph 4). This can be delivered as an Enterprise Platform, a standalone Terminal Solution, or as an API Engine that comes with more than 300 JSON API calls, including services for specific user segments such as asset-liability management.

It has been reported that some users have obtained financial advice through the use of ChatGPT, a large language model (LLM). While this is a testament to the potential of LLMs, it's important to recognize the risks, such as the issue of "hallucination". For example, the *Washington Post* has reported that ChatGPT fabricated a false incident of college sexual harassment and attached a real person's name to the case [12]. Therefore, it's essential to develop robots for applications that require domain expertise differently from simply building an extremely smart natural language interface, i.e. the data being fed to train the AI-bot should come from an expert system consistent with rigorous professional standards in the first place. This background context may help explain the unique parameters faced by the computational problem of interest. Firstly, we begin by collecting an extensive amount of data that includes all potentially relevant structured and unstructured data. Secondly, the dataset is continuously updated in real time, including both updates to existing data elements and the addition of relevant new elements to the data table. Finally and most importantly, it is essential to extract relevant information from this massive database in an explainable manner, as human reviews will be necessary to ensure that fiduciary duties are met. While AI is an exceptional tool for in-sample analysis, it has been problematic for out-of-sample analysis. One user objective is to conduct regular human reviews to ensure that

979-8-3503-4323-6/23/$31.00 ©2023 IEEE
DOI 10.1109/QCE57702.2023.00054

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

the AI-bot will minimize if not fully eliminate human-friendly but professionally non-sensical outputs.

B. Problem Statement

As part of our continuous innovation, the company has formed successful partnerships with Nvidia and IBM Quantum. This paper is about solving two applications in finance using state-of-the-art hardware, as follows:

1) Problem 1: We aim to create an indicator vector, in which each asset is represented as an element in the vector, 1 for present and 0 for not present, that will yield a portfolio under a risk-return metric such as the highest Sharpe ratio. It should be noted that, in practice, objective functions more sophisticated than the textbook Sharpe ratio are often chosen for managing institutional investment portfolios. The first step in solving the problem, known as finding portfolio cuts, sounds intuitively straightforward for a small number of exemplar assets, but its complexity can be prohibitive when applied to real-world, industrial-sized problems. Let's assume that A is the set of all admissible assets. Our goal is to find a subset of A, denoted by $\{x_1, x_2, \ldots, x_n\}$, such that

- the weight attributed to each asset is great than or equal to zero,
- the weights w_i associated with each asset x_i sum up to one, i.e. $\sum_{i=1}^{n} w_i = 1$
- the number of assets chosen is smaller than some user-prescribed maximum $n < N$, and
- while a given objective function $f(w_1, w_2, \ldots, w_n)$ is at its maximum or within a confidence boundary.

One way to solve this problem is by stepping w_i on a certain discrete grid with increments of (say) 0.5%, starting from zero. This is computationally time-consuming and not tractable for a large set of available assets to choose from. An alternative way to think of the problem in a Graph Theory sense and essentially to seek a solution in which the objective is to cut the set A into two disjoint subsets so that one subset $\{x_1, x_2, \ldots, x_n\}$ can maximize the objective function $f(w_1, w_2, \ldots, w_n)$ or reach within a confidence boundary. To illustrate the complexity of the problem, if $N = 500$ (e.g. that would be the case for the S&P 500 stock components), the number of combinations to split the vertices into two subsets is $C = 2^{499} = 1.6 \times 10^{150}$, and that is even before we optimize the objective function by adjusting the weights of the assets in the chosen subset! However, contrary to the computationally intensive approach sketched in the previous description, a computationally efficient approach to solve a similar problem using Graph Theory has been described in Chapter 13 of *Data Analytics on Graphs* [9].

In practical investment management, institutional investors often request a tailored portfolio that closely replicates the sector allocation of a given benchmark index. By adopting such a strategy, investors can establish a long position in the customized portfolio and a corresponding short position in the index to capture the so-called "alpha" or outperformance. For instance, assuming an average of 20 assets per sector, the minimum search required to construct a portfolio that matches the sector composition of the S&P 500 index would entail exploring $C = 25 \times 2^{19} = 1.3 \times 10^7$ possible combinations, rather than an intractable number of $C = 1.6 \times 10^{150}$ combinations. However, it is also well known among seasoned investment managers that the use of fundamental sector definitions alone may lead to poorly defined sectors, thereby necessitating the identification of an efficient approach to cut the asset universe A into more effectively-defined sectors. In this regard, the key to addressing Problem 1 is to develop a method that combines fundamental sector definitions with statistical characteristics to achieve an effective sector cut of the asset universe.

2) Problem 2: Once we have identified the sectors, the returns of each asset within the sectors can be driven by a set of factors such as $\{f_1, f_2, \ldots, f_m\}$. Typically, each sector or sub-sector contains about an average of 20 or so assets while the factor set can easily exceed a few hundred data points (say from the asset's financial data). Allowing all such factors to be used in any alpha selection model will result in the well-known problem of model overfitting, as in one may find a model with good adjusted R^2 but with relatively weak *ex-ante* predictive power. The classic solutions are a) LEAPS regression to try all possible combinations of factors, b) certain "branch and bound" style heuristic to select a subset of factors or c) advanced graph neural network algorithms that will incorporate the grouping information, an advanced concept that we will revisit in Section III. Our initial goal is to identify a small collection of distinct factors to form a model. However, these are pure numerical methods. Using these methods, the computer is not benefiting from any domain knowledge, such as how Dividends and Earnings Per Share give similar information so we only need to pick only one factor from these two. In fact, using very similar factors within a single model may result in linear dependency among the factors, which may hurt the predictive powers of the resulting model and create numerical issues. In other words, the desirable approach is to choose at most one factor from each subset $\{f_1, \ldots, f_{m_1}\}, \{f_{m_1+1}, \ldots, f_{m_2}\}, \{f_{m_2+1}, \ldots, f_{m_3}\}, \cdots$ where $m_i <= m$. The problem is how to cut the set $\{f_1, f_2, \ldots, f_m\}$ into the appropriate subsets based on both the fundamental grouping information of the factors along with other statistical characteristics such as the historical correlation coefficients observed among the factors.

Problems 1 and 2 can be solved with similar techniques using different inputs. To simplify our illustration, the discussion below will focus on solving Problem 1.

C. Portfolio Graph Cuts

For illustration, we use $n = 1000$ from the universe of Russells 1000 components. If we invest in the entire universe of roughly 1000 stocks scaled by market capitalization, by definition such a portfolio becomes a market-capitalization index and there will be no out-performance, which is uninteresting to many investors. For each portfolio cut, we try to produce a subset with similar characteristics, relative to traditional Global Industry Classification Standard [13] (or "GICS") which are defined based on fundamental criteria alone. As a specific example,

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

Fig. 1. Scientifically-validated Track Record.

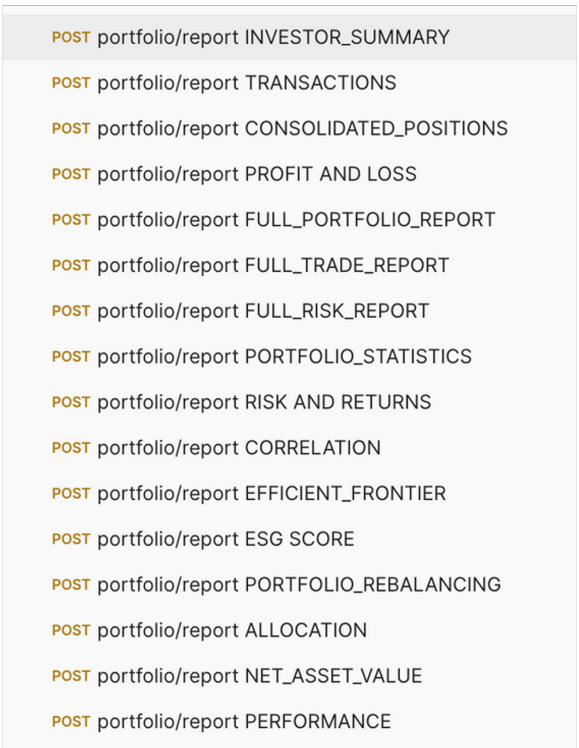

Fig. 2. Samples of Institutional Investment Reports.

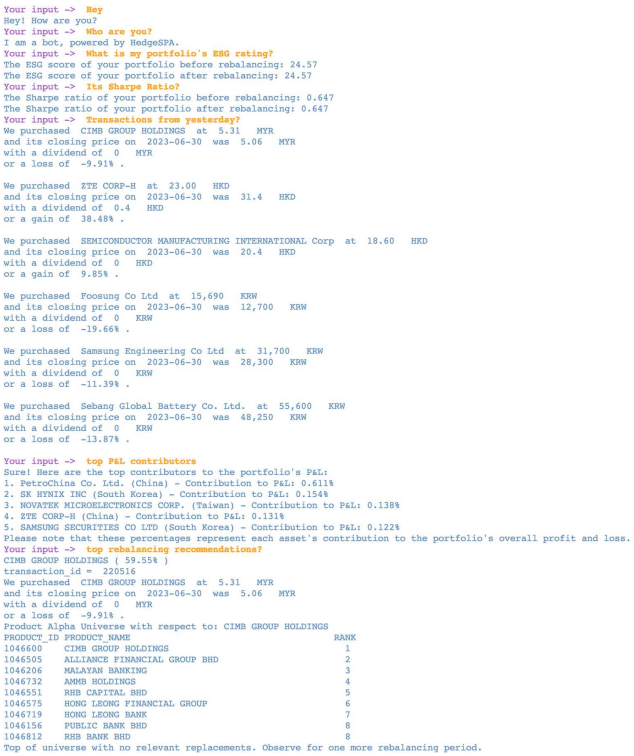

Fig. 3. Sample Screen from Conversational AI-Bot.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

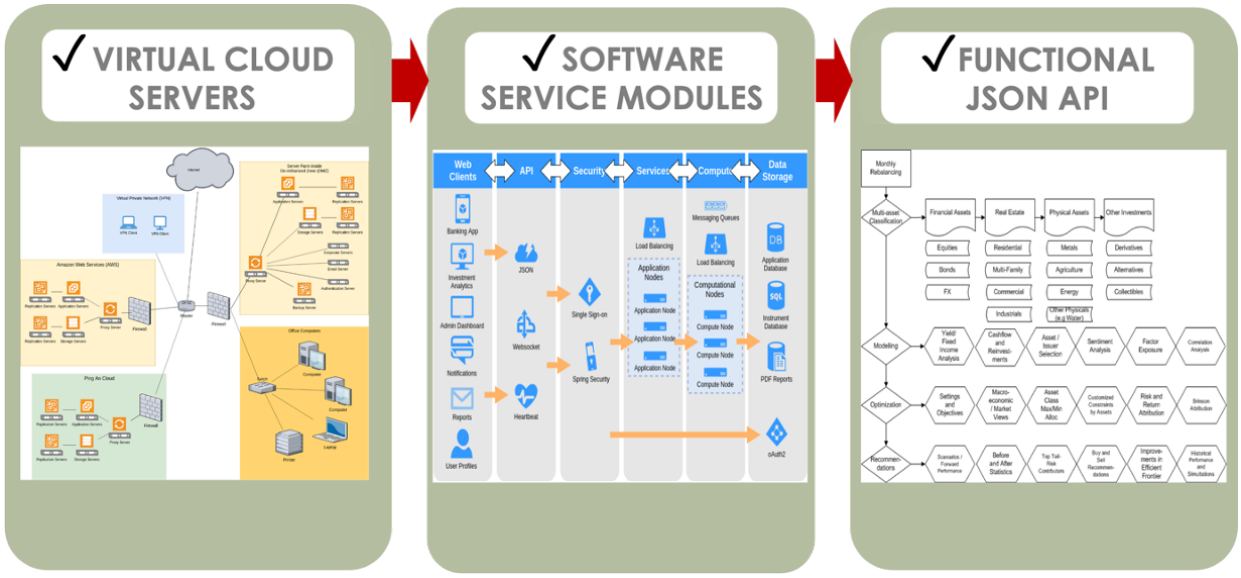

Fig. 4. Multi-layered Software Security Architecture.

Tesla is classified as automotive stock based on fundamental criteria, although many may argue that its asset prices behave like those of technology stocks. Grouping Tesla with other automotive stocks may only result in Tesla being chosen as an "outperformer" in the automotive sector when in fact such a classification is a misspecification.

The alternative is to invest in the best-performing stock(s) in each cut so that we create out-performance relative to the underlying index while preserving the diversification benefits of including all the index countries and/or sectors. Next, we describe the precise algorithm required. We use the notation of an n-node undirected graph $G = (V, E)$ where V denotes a set of vertices or nodes with $|V| = n$, and E denotes a set of edges with edge weights $w_{ij} > 0, w_{ij} = w_{ji}$ for $(i, j) \in E$, which is defined as the absolute price correlation coefficient in this case, i.e. $|\rho_{ij}|$, or $w_{ij} = \frac{|\sigma_{ij}|}{\sqrt{\sigma_{ii}\sigma_{jj}}} = |\rho_{ij}|$, where $\sigma_{ij} = cov(r_i(t), r_j(t))$ is the covariance of asset returns between assets i and j. The degree matrix, $\mathbf{D} \in \mathbb{R}^{N\times N}$, is a diagonal matrix with its elements defined as $D_{mm} = \sum_{n=1}^{N} w_{mn}$. Then, the $N \times N$ graph Laplacian matrix, $\mathbf{L} \in \mathbb{R}^{N\times N}$, defined as $\mathbf{L} = \mathbf{D}\text{-}\mathbf{W}$, serves as an operator for evaluating the curvature, or smoothness, of the graph topology. Now we group the n-vertex market graph, $G = \{V, E\}$, into $K = 2$ disjoint subsets of vertices, $V_1 \subset V$ and $V_2 \subset V$, with $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$. A cut of this graph, for the given clusters, V_1 and V_2, is equal to a sum of all weights that correspond to the edges which connect the vertices between the subsets, V_1 and V_2, i.e,

$$Cut(V_1, V_2) = \sum_{m\in V_1} \sum_{n\in V_2} w_{nm}.$$

The normalized cut is regularized by an additional term to enforce that the subsets V_1 and V_2 should be simultaneously as large as possible:

$$CutN(V_1, V_2) = \left(\frac{1}{N_1} + \frac{1}{N_2}\right) \sum_{m\in V_1} \sum_{n\in V_2} w_{nm}$$

Further, it can be shown that if an indicator vector is defined as:

$$\text{x(n)}= \begin{cases} \frac{1}{N_1}, & \text{for } n \in V_1 \\ -\frac{1}{N_2}, & \text{for } n \in V_2 \end{cases}$$

then the normalized cut, i.e. $CutN(V_1, V_2)$, can be found by the Rayleigh quotient of $\mathbf{L}$ and $\mathbf{x}$, i.e.

$$CutN(V_1, V_2) = \frac{\mathbf{x}^T\mathbf{L}\mathbf{x}}{\mathbf{x}^T\mathbf{x}}.$$

The indicator vector, $\mathbf{x}$, which minimizes the normalized cut also minimizes $CutN(V_1, V_2)$. We rewrite this minimization problem, for the unit-norm form of the indicator vector, as $\min_{\mathbf{x}} \mathbf{x}^T\mathbf{L}\mathbf{x}, s.t. \quad \mathbf{x}^T\mathbf{x} = 1$, which can be solved through the eigenanalysis of $\mathbf{L}$, i.e. $\mathbf{L}\mathbf{x} = \lambda_k\mathbf{x}$. After neglecting the trivial solution $\mathbf{x} = \mathbf{u}_1$ for $k = 1$, since it produces a constant eigenvector, we arrive at $\mathbf{x} = \mathbf{u}_2$ for $k = 2$. Therefore, the membership of a vertex n in either the subset V_1 or V_2 is uniquely defined by the sign of the indicator vector $\mathbf{x} = \mathbf{u}_2$, i.e.

$$\text{sign(x(n))}= \begin{cases} 1, & \text{for } n \in V_1 \\ -1, & \text{for } n \in V_2 \end{cases}$$

Notice that scaling of $\mathbf{x}$ by any constant would not impact the clustering of a vertex n into subsets of V_1 or V_2. It is straightforward to generalize to $K \geq 2$ disjoint sub-graphs through the method of repeated bisection. For instance, $k = 4$

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

portfolio cuts generate $2^4 = 16$ leaves of the market graph, with each statistically-derived sector averaging 62.5 assets for $n = 1000$. Since the size of each portfolio cut is not evenly distributed, we will target the maximally acceptable size of a proper portfolio cut to be 125, or twice the number of average assets in each cut. This heuristic criterion will turn out to be very helpful in solving practical industry problems.

D. Mixing Selection Criteria in Graph Laplacian Matrix

An alternative definition of edge weight v_{ij} is given by

$$v_{ij} = \begin{cases} 1, & \text{if assets } i \text{ and } j \text{ are in the same GICS sector} \\ 0, & \text{if assets } i \text{ and } j \text{ are in different GICS sectors.} \end{cases}$$

GICS [13] is an industry-standard taxonomy used to describe asset sectors such as Energy, Industrials, Information Technology, etc. Next, we use the following linear combination of v_{ij} and w_{ij}, i.e. $\kappa \times v_{ij} + (1-\kappa) \times w_{ij}$, to construct the $N \times N$ graph Laplacian matrix $\mathbf{L}$. When $\kappa = 1$, the graph Laplacian contains only the GICS sector information of the assets, so the portfolio graph cuts can only return the partitioning by sectors. Our numerical experiments show that when $\kappa = 0.9$, the graph Laplacian is still dominated by the GICS sector information of the assets, but the graph cuts are beginning to be influenced by how certain asset prices may strongly correlate with each other. When $\kappa = 0.3$, the graph Laplacian is now driven by both the GICS information and absolute correlation among the assets. Such portfolio cuts may allow a mix of sectors with mostly "new economy" stocks mixed with a handful of "old economy" stocks.

E. Criteria for Intuitive Portfolio Cuts

Since at this point, GICS sector information carries only 30% of the edge weights, for the remaining 70%, we can mix in as much information as desirable when defining the edge weights, such as certain correlation measures of data reported by the candidate companies. The practical issue is whether the results generated will be reasonably intuitive, which may include the following criteria:

1) The largest portfolio cuts should contain no more than 125 assets;
2) Either there are no "orphan" cuts (cuts with only one or two assets) or there should be effective ways to eliminate them by recombining leaves; and
3) The methodology should include "intelligence" to group companies such as Tesla with the technology universe based on statistical properties of asset returns.

For a universe of 1000 assets, the eigenvectors associated with the smallest 50 or so non-zero eigenvalues are reasonably smooth (let's call the set S), and the computer can choose a subset of S to do the cuts so that it keeps progressing on the leaves until the largest cut contains no more than 125 assets. Numerical results suggest that we can do so with 17 eigenvectors chosen from a set S of 34 eigenvectors. We have also eliminated one out of any two neighboring eigenvectors with less than 10 variations in signs, which will almost surely result in a trivial if not an identical cut. Choosing 17 eigenvectors out of a set of 34, i.e. $\binom{34}{17}$, can result in over 2 billion total combinations. The goal is to sample only a few hundred combinations so that the computer will satisfy the solution objective as stated above while generating cuts that reflect either sector characteristics or asset price correlations, or ideally a combination of both.

If we run (say) 2000 samples (or 0.0001% of the 2 billion combinations), we may get more effective portfolio cuts, but doing so requires generating much larger sampling sets. Generating correlated random vectors is complicated on a classical computer but trivial on a quantum computer. Do note that any complicated matrix computations are already completed at the point of the random sampling, so the remaining steps are about "scoring" the cuts, which are all simple arithmetics with linear complexity. In addition, only the eigenvector information needs to be sent to the random sampling engine at that point. The next section will discuss how we can achieve "quantum leaps" in computational efficiency by using a quantum computer to solve what would have been a time-consuming problem using classical computers.

F. Quantumized Variant to the Algoirthm

Based on the algorithm described above, we use $n = 1000$ and target 16 cuts. Each sector has 62.5 assets on average. Assuming that we have a reasonably robust model for finding asset alpha, we limit the problem to picking 3 assets from (say) the top 5 identified in each cut. In that case, the maximum number of portfolio combinations to check will be $\binom{5}{3}^{16} = 10^{16}$, which is still a very large number but is a significant reduction in the order of magnitude from the initial combinations of 1.6×10^{150} as stated in the very beginning of this paper. Thus, we have aggressively reduced the dimensionality of the problem by controlling the number of desirable assets allowed in each portfolio cut. In each of the target cuts, we choose only 3 assets from the top 5 assets screened out by the alpha model to create a portfolio of roughly 48 stocks, which is a portfolio of a size consistent with industry practice. Such a problem is at least computationally tractable using modern computers.

This problem bears a significant resemblance to the more generalized MAXCUT and MAXCUT-SAT problems. Their solutions have quantum analogs [4]. Essentially, a quantum computer can produce a simultaneous random vector far more efficiently than a classical computer can without the typical time-consuming matrix inversion step to compute Cholesky decompositions to generate correlated random vectors. As a result, we can use the quantumized random vector as a tool for performing extremely fast random sampling. The key to practical application is to find reliable bounds for convergence. As a future R&D project, the authors are keen to understand whether the well-known bounds derived for solving combinatoric optimization problems on classical computers can be similarly applied to their quantum-equivalent solutions [2].

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

To give a more specific example, MAXCUT is an NP-complete problem defined as follows. As before, the problem is a 1000-node undirected graph $G = (V, E)$ where V denotes the set of vectors or nodes with $|V| = n$, and E denotes a set of edges with edge weights $w_{ij} > 0, w_{ij} = w_{ji}$ for $(i, j) \in E$, The ultimate goal is to find partitions of the original set V into n subsets, V_1, V_2, …, V_n, where each subset contains fewer than the maximally allowed 125 assets. As mentioned earlier, this is done by applying the method of repeated bisections until the leave of each cut in a market graph reaches the desired size. The cost function to be minimized in each cut is the sum of weights of edge-connected points in the two different subsets (crossing the cut), Assign $x_i = 0$ or $x_i = 1$ to each node i to *maximize* the global objective function $\hat{C}(x) = -\sum_{i,j} w_{ij} x_i (1 - x_j)$ or more generally as

$$C(x) = -\sum_{i,j} w_{ij} x_i (1 - x_j) - \sum_i w_i x_i$$

to incorporate both quadratic and linear terms in the objective function C. To map this algorithm to a quantum circuit, map the objective to an Ising Hamiltonian: $x_i \to (1 - Z_i)/2$, where Z_i is the Pauli Z operator with eigenvalues ± 1:

$$C(Z) = -\sum_{i,j} \frac{w_{ij}}{4} (1 - Z_i)(1 + Z_i) - \sum_i \frac{w_i}{2} (1 - Z_i)$$

$$= \frac{1}{2} \sum_{i<j} w_{ij} Z_i Z_j + \sum_i w_i Z_i + const.$$

Thus, the weighted MAX-CUT problem is equivalent to *minimizing* the following Ising Hamiltonian:

$$H = -\sum_i w_i Z_i - \sum_{i<j} w_{ij} Z_i Z_j$$

A solution solving for this functional form has been implemented by the IBM Qiskit optimization module which can generate the Ising Hamiltonian for the objective function C [14]. The classical computation as described earlier has taken hours if not days when performed on a state-of-the-art 8-core computer with RISC architecture. Our goal is to show that we can reduce the graph-cut computation for the real world, industrial-sized problems to minutes if not seconds by using a quantum computer on the cloud, so that the new portfolio results can be obtained in a matter of minutes if not seconds in each rebalancing cycle for an institutional user.

II. Computational Experiments

Some preliminary computational findings were presented in the initial research manuscript. The objective of this paper is to establish the objective evidence of the accelerated performance potential ahead of allocating substantial resources toward quantum computing time and expenses.

By establishing a certain baseline comparison between classical and quantum methodologies on problems of increasing complexity, we aim to quantify the projected speed and scale advantages of quantum computing for our specific domain applications. Without empirically deriving the quantum speedup for our targeted use cases, investments in quantum computing cycles may be viewed by certain institutional users as premature and/or misdirected.

Through controlled experiments using a hybrid classical-quantum workflow, we will determine the quantum simulation crossover point, if one exists, for optimally solving our classes of computationally intensive problems. Only once this threshold has been substantively proven through measurable results will a case be made to pursue quantum computing at an industrial scale.

A. Test Environment

A dataset consisting of 945 large capitalization United States equities with 832 closing prices was collected on trading dates from January 2, 2020, to April 21, 2023. This sample size and date range aim to ensure non-singular covariance matrices for a typical universe of no more than 500 securities, thereby avoiding potential numerical instability issues. The following is a sample record:

Description	**Field Name**	**Value**
Security identifiers	product_id	1009079
Security names	product_name	"Apple Inc"
Close of business data	cob_date	2020-03-03
Closing price	CLOSE	72.33
Industry sector ID	SECTOR_ID	45
Sub-industry sector ID	sub_sector_ID	452030

This test environment will be used to evaluate the performance, accuracy, and insights generated from algorithms trained and inferred on this data using both classical and quantum machine learning approaches. Head-to-head comparisons will determine where quantum computing offers a meaningful advantage for this domain using a typical sample dataset.

B. Experiment Design

1) Select an initial subuniverse of 12 securities from the sample dataset.
2) Perform the graph cut algorithm available from IBM Qiskit [14] three times on this subuniverse using:
   - Brute Force Approach (classical computing baseline by computing all possible combinations)
   - Eigensolver (enhanced approach on a classical computer by using eigenvalue optimization)
   - Quantum Simulator (simulated quantum computing approach)
3) Increase the number of securities in the subuniverse and repeat step 2 to determine the performance crossover point, if any, between classical and quantum simulation methodologies.
4) Normalize the data by subtracting the initial value (of 12 securities) from each data point so they evaluate relative to a common baseline of zero for easier regression modeling and comparison:
   - $\frac{Number_of_Stocks_i}{12} - 1$;
   - $\ln(\frac{Time_i}{Time_{12}})$.
5) Finetune the calculations in the following manner:

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

- Initially, `aer_simulations_statevector` was deployed as the backend quantum simulator, which ran out of memory because this older implementation was known to allocate the total memory for the quantum state vector upfront, causing the virtual machine (with 8GB of memory) to run out of memory and crash. Thereafter, we used the more up-to-date `qasm_simulator` which allocated memory only as needed to avoid the memory issue, enabling larger experimental runs.
- We could use the `ibmq_qasm_simulator` backend quantum simulator which allocates the QASM simulator from the IBM cloud, but doing so introduces queueing delays. Thus, we save that approach for larger experiments in the future when we will run out of local memory. By comparison, the `qasm_simulator` leverages available local memory (8GB) under our direct control.

C. Results

We establish a rigorous, evidence-based process for determining where and how quantum computing may offer meaningful speed and scale benefits for graph-based financial algorithms on our sample dataset, as shown in Figures 5, 6 and 7 on the next page.

For a production implementation using real-life industry data, these are the recorded runtimes for the Hamiltonian step `max_cut.sample_most_likely` of the computation from the IBM Qiskit library:

Number of Assets	Ellapsed Time for Hamiltonian Step (Seconds)
10	0.34
12	0.34
15	0.64
18	2.44
21	27.79
24	325.94
25	730.38

The purpose of this test is to gain insight into how a production implementation would perform on typical hardware. At present, we are only able to run the test on a machine with a typical `t2.large` allocation on AWS, which has 8 GB of memory and 2 vCPUs. The test can only be run up to $n = 25$ before the machine runs out of memory while attempting to allocate an array that exceeds a certain size. We are aware of this limitation and we plan to address it in the future by developing customized algorithms using `cuQuantum` on Nvidia. This will enable us to avoid allocating unnecessarily large arrays for this computation and allow us to run the test on larger values of n.

D. Analysis

1) The absolute runtime of the algorithm is less important for this analysis than its growth rates, particularly for problems of bigger and bigger size. It is interesting that at $n = 24$ the absolute time required for solving the problem on the quantum simulator begins to catch up with that for the brute-force algorithm. However, this comparison is not important since the runtime on a real quantum computer will be thousands if not millions of times faster than that on a quantum simulator.
2) Interestingly enough, the growth rate of the eigensolver approach is not a huge improvement over the brute force approach. This is likely associated with the inversion of a very large matrix, as well as the exponential search method after the matrix inversion.
3) The quantum approach can be roughly modeled by:

$$\ln(speed/base) \sim 4(N-12)$$

$$\implies \qquad speed \sim base * e^{4(N-12)}$$

which is a significant improvement over $O(e^{9N})$ for the other two methods.
4) Regarding the runtime performance of the system, while we understand that the absolute runtime is not critical, we are still working towards improving its performance on IBM Qiskit, so that it can fully justify the expensive subscription of a quantum back-end. However, Nvidia has also implemented a version of MAXCUT on a DGX machine, as mentioned in [15]. Although the sample code is available online, we have found some compatibility issues between Nvidia's `cuQunatum` library and the currently supported version of `Tensorflow` (which the sample code has a dependency on) running on AWS SageMaker. Since this is hardware-dependent, the debugging process is time consuming. Nevertheless, we remain hopeful that some additional results will be available by the time of the IEEE QCE23 conference.

E. Goal

For any computational problem with a combinatorial explosion in complexities, exponential growth must eventually overwhelm physical resources and practical viability, no matter what is the absolute processing speed. Our focus thus rightly centers on finding the most efficient quantum algorithms to solve problems that are intractable on classical computers with no known polylogarithmic or polynomial time solutions. The eventual migration from a quantum simulator to a real quantum computer will cut down computational time by thousands of times if not in the millions.

Our challenge centers on financial datasets commonly consisting of a few hundred stocks (such as $n = 500$) so we will need access to a quantum computer with a high number of qubits. The current state-of-the-art hardware is currently at 127 to 433 qubits. We propose the following approaches to reducing exponential time complexities to more tractable levels for n in the hundreds:

- Use the $O(n^3)$ eigenvalue decomposition to reduce the problem to its spectral subspace. Computing only the top 10 to 30 eigenvalues and corresponding eigenvectors, which capture the majority of variability while requiring far fewer computational resources.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

Stocks	12	13	14	15	18	21	24	SLOPE (ln(SpeedUp) vs (Stock/Base - 1))
Stocks/Base	1	1.08	1.17	1.25	1.5	1.75	2	
Stocks/Base - 1	0.00	0.08	0.17	0.25	0.50	0.75	1.00	
Brute Force (seconds)	**0.9**	**1.42**	**2.86**	**5.42**	**58.62**	**666.93**	**7050.94**	
Brute Force Speedup	100%	158%	318%	602%	6513%	74103%	783438%	
ln(Brute Force Speedup)	0.00	0.46	1.16	1.80	4.18	6.61	8.97	9.154
Eigensolver (seconds)	**1.03**	**1.3**	**2.27**	**4.33**	**46.84**	**671.77**	**8524.13**	
Eigensolver Speedup	100%	126%	220%	420%	5204%	74641%	947126%	
ln(Eigensolver Speedup)	0.00	0.23	0.79	1.44	3.95	6.62	9.16	9.512
QASM Sim (seconds)	**151.82**	**196.01**	**314.75**	**456.91**	**1281.00**	**2875.76**	**9222.86**	
QASM Sim Speedup	100%	129%	207%	301%	844%	1894%	6075%	
ln(QASM Sim Speedup)	0.00	0.26	0.73	1.10	2.13	2.94	4.11	4.065

Fig. 5. Computational Results in Table Form.

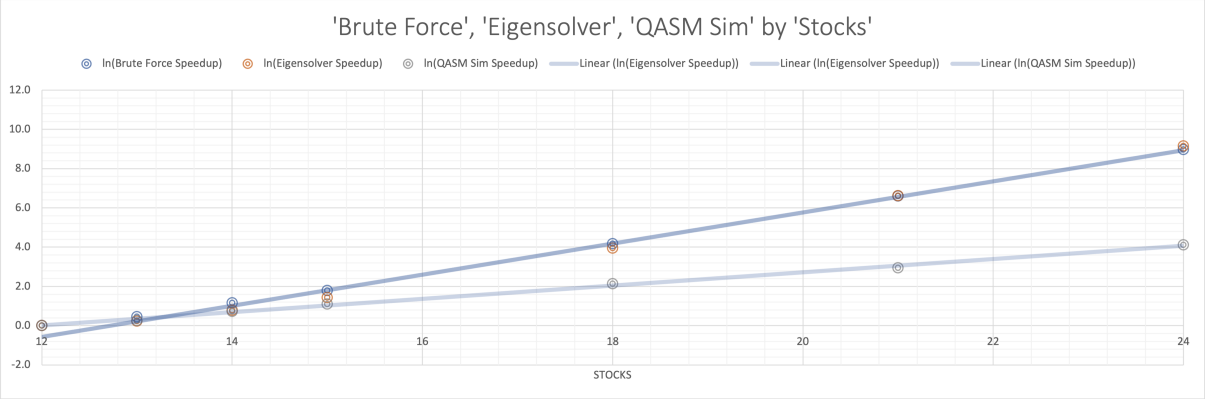

Fig. 6. Graph of $\ln(SpeedUp)$ in Linear Scale with with Linear Trendlines.

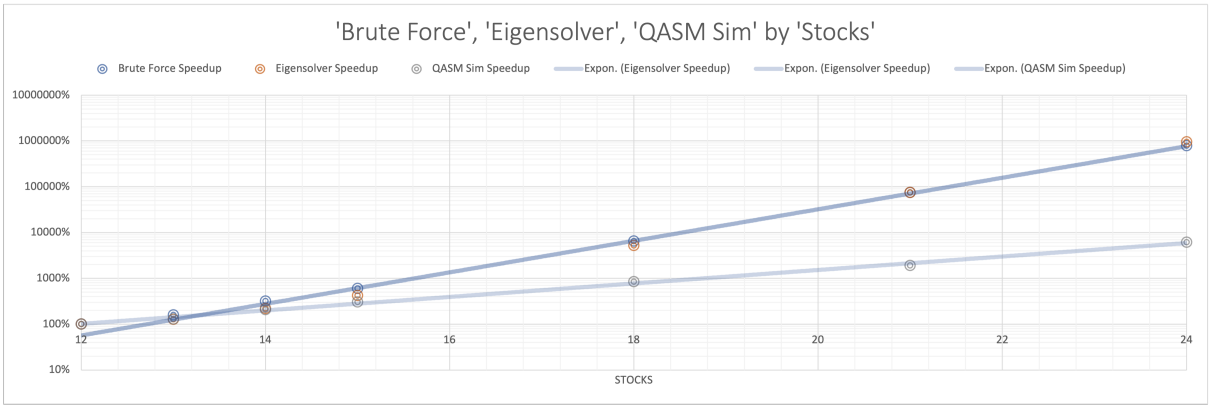

Fig. 7. Graph of $SpeedUp$ in Logarithmic Scale with Exponential Trendlines.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

- Working within this projected subspace spans a space of size m, where m is typically 10 to 25 instead of n in the hundreds. While explicit search and optimization in this m-dimensional space still have the complexity of 2^m, a problem with $m = 25$ is still far more tractable than one with $n = 500$. This subspace-constrained approach yields a "smart search" instead of an explicit, unintelligent search of the entire n-dimensional space.
- Solving the enhanced problem within this m-dimensional subspace has a complexity closer to $O(n^3) + O(2^m)$ instead of $O(2^n)$. Put differently, even if a quantum computer speeds up the computation by 1 billion times, we are still stuck with $O(2^{500}/1,000,000,000) = 3.3 \times 10^{141}$. For $m = 25$, $O(2^m)$ is 33.6 million, which is tractable. Additional compression techniques can be applied to shrink m as much as possible but still achieve a computationally tractable solution.

III. Future Research

Besides understanding the well-known bounds derived for combinatoric optimization that can be applied to their quantum-equivalent solutions [2], we have identified four follow-on R&D projects, as follows:

1) Firstly, we propose smart searches within compressed, reduced subspaces as a means of mitigating constraints instead of relying on speedup from ever more expensive hardware. Denote $\mathscr{F}(\mathbf{x})$ as the forward transform and $\mathbf{x} = \mathscr{F}^{-1}(\mathscr{F}(\mathbf{x}))$ as the inverse transform. Projecting a high-dimensional problem of intractable complexity onto a lower-dimensional manifold can be described as $maxcut_n\mathbf{x} \sim maxcut_n\mathscr{F}^{-1}(\mathscr{F}(\mathbf{x})) \sim \mathscr{F}^{-1}(maxcut_{n\to m}\mathscr{F}(\mathbf{x}))$. $\mathscr{F}(\mathbf{x})$ is typically a linear transform so the approximate solution remains sufficiently representative as long as certain smoothness properties of the objective function are met, but the lower-dimensional problem is expected to be far more tractable from a computational point of view. Deriving an error bound similar to that provided by Christofides [2] will be an exciting area of future research. Given the nature of our financial problems, a reasonable approximation is often very useful, since we can control m based on the type of hardware available, but we often do not control how large n is, so an inadvertent choice by the user may lead to out-of-memory crashes.
2) Secondly, our method of computing sector cuts is based on $\kappa \times v_{ij} + (1-\kappa) \times w_{ij}$. If we drop the fundamental information completely (i.e. setting $\kappa = 0$), then the sector cut computation is practically identical to the Implied Graph calculation in Proposition 2.2 in [6], which is based on a Cholesky decomposition of the Graph Laplacian. An Implied Graph is defined as the best-fit graph that can be generated from reversed optimization on a given set of data under the specified degree of freedom. If the Graph Laplacian is not symmetric (i.e., as in a directional graph), then a Singular Value Decomposition can also be applied. Either way, Implied Graphs offer a visually-friendly method to represent very large datasets (e.g. the full factor set) so that human reviews can be conducted more readily. For instance, these Implied Graphs can be very useful in modeling complex socioeconomic processes that cannot be modeled by a system of closed-form equations.
3) Thirdly, instead of running LEAPS regressions to predict asset returns, we can use the Implied Graph information to construct the graph embedding in the training of a GNN, where:
   - In the *Aggregate* step, the hidden states of immediate neighbors of node i are aggregated, i.e. $a_i^l = Aggregate\left(h_j^{l-1} : j \in \mathscr{N}(i)\right)$.
   - In the *Combine* step, the node embedding for the l-th GNN layer is generated, i.e. $h_i^l = Combine\left(h_i^{l-1}, a_i^l\right)$.

   The node embedding is represented by h_i^l, where l denotes the number of layers of aggregation, and $\mathscr{N}(i)$ denotes the set of immediate neighbors of node x_i. We can define the first layer of embedding based on the sector or factor cut and its immediate neighbors using members of that cut, and the second layer based on adjacent leaves of the bigger graph and so on. This approach allows for l layers of aggregating the node embedding to be fine-tuned, while weight matrices are not explicitly used in the *Combine* step so that the overall training can use all information available, and their relative importance or relevance only needs to be decided in the training step.
4) Finally, since it is not feasible to retrain the GNN continuously as real-time financial data streams and updates constantly, it will be helpful to devise a heuristic so that changes in outputs to the GNN can be estimated under minor changes in input data. Going back to the notations used in Section I, Subsection B, denote our objective at time t as the functional $F(\mathbf{g}(\mathbf{r}))|_t = max_{w_i} f|_t(w_1, w_2, \ldots, w_n)$, where $\mathbf{g}$ denotes the GNN used to choose the subset of desirable assets $\{x_1, x_2, \ldots, x_n\}$ and $\mathbf{r}$ denotes the vector of input data used to train the GNN. Then, the Taylor Series expansion [3] about the functional $F|_{t_0}$ is given by:

$$F|_{t_1} \approx F|_{t_0} + \int d\mathbf{r} \frac{\delta F}{\delta \mathbf{g}(\mathbf{r})}\bigg|_{t=t_0} [\mathbf{g}(\mathbf{r}_{t_1}) - \mathbf{g}(\mathbf{r}_{t_0})]$$

   where $\frac{\delta F}{\delta \mathbf{g}(\mathbf{r})}\big|_{t=t_0}$ can be estimated from changes in the GNN outputs from time t_0 to t_1. In practice, this approximation only works with small changes in the input data, so the impact of a particular data element r_j on the GNN output can be estimated by tracing its associated nodes and edges in the GNN. If the edge weights associated with a particular r_j are below certain thresholds, we can simply skip the estimation of the impact on $\frac{\delta F}{\delta \mathbf{g}(\mathbf{r})}\big|_{t=t_0}$. On the other hand, if both the edge weights associated with a particular r_j and therefore the resulting impact on the GNN output $[\mathbf{g}(r_j(t_1)) - \mathbf{g}(r_j(t_0))]$ are significant,

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.

we can design a heuristical rule to trigger the retraining of the GNN. The precise methods of estimating these functional derivatives so that it is more efficient to estimate them numerically instead of retraining the GNN will be an exciting topic for future research.

Please note that there are some areas of future research which have the potential to yield significant rewards for any real-life applications of quantum algorithms in investment analytics, as well as related fields such as real-time process control or medical informatics. It is expected that these areas of research will have a profound impact on the future of asset management and other industries, as the speed of processing and volume of data that can be handled are likely to disrupt existing capabilities.

REFERENCES

[1] J. Bruna, W. Zaremba, A. Szlam, and Y. Lecun, "Spectral networks and locally connected networks on graphs," in: *Proceedings of ICLR*, 2014.

[2] Nicos Christofides, "Bounds for the Travelling Salesman Problem", *Operations Research*, Vol. 20, No. 5 (Sep. - Oct., 1972), pp. 1044-1056 (13 pages). https://www.jstor.org/stable/169166?seq=1.

[3] Matthias Ernzerhof, "Taylor-series expansion of density functionals", *Physical Review A* 50, 4593, 1 December 1994. https://journals.aps.org/pra/abstract/10.1103/PhysRevA.50.4593.

[4] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann, "A Quantum Approximate Optimization Algorithm", MIT-CTP/4610, MIT Center for Theoretical Physics, 2014. https://arxiv.org/pdf/1411.4028.pdf.

[5] W. Bernard Lee, and Anthony G. Constantinides, "Quantumized Graph Cuts in Portfolio Construction and Asset Selection", forthcoming in *Springer-Nature Transactions on Computational Science and Computational Intelligence*, 2023.

[6] Yaguang Li, Rose Yu, Cyrus Shahabi, Yan Liu, and Yiming Yang, "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting," *AAAI Conference on Artificial Intelligence*, 2018, pp. 6295-6302. https://arxiv.org/abs/1707.01926

[7] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst, "The emerging field of signal processing on graphs: extending high-dimensional data analysis to networks and other irregular domains", *IEEE SPM 30 (2013)*, pages 83-98.

[8] Abishek Sriramulu, Nicolas Fourrier, and Christoph Bergmeir, "Adaptive dependency learning graph neural networks", *Information Sciences*, Volume 625, May 2023, Pages 700-714.

[9] Ljubiša Stanković, Danilo Mandic, Miloš Daković, Miloš Brajović, Bruno Scalzo, Shengxi Li and Anthony G. Constantinides, "Chapter 13: Portfolio Cuts", *Data Analytics on Graphs Part III: Machine Learning on Graphs, from Graph Topology to Applications*, Foundations and Trends® in Machine Learning: Vol. 13: No. 4, pp 499-511. http://dx.doi.org/10.1561/2200000078-3.

[10] Fei Tan, Lili Zhou, Junwei Lu, Hongzheng Quan, Kaiyue Liu, "Adaptive quantitative control for finite time synchronization among multiplex switched nonlinear coupling complex networks", *European Journal of Control (2023)*, pages 23-37.

[11] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, Maosong Sun, "Graph neural networks: A review of methods and applications", *AI Open 1 (2020)*, Pages 57-81.

[12] "ChatGPT invented a sexual harassment scandal and named a real law prof as the accused", *Washington Post*, April 5, 2023. https://www.washingtonpost.com/technology/2023/04/05/chatgpt-lies/

[13] "Global Industry Classification Standard", *Wikipedia*. https://en.wikipedia.org/wiki/Global_Industry_Classification_Standard.

[14] "Max-Cut and Traveling Salesman Problem", *Qiskit Tutorials*, IBM Quantum Lab. https://qiskit.org/documentation/tutorials/optimization/6_examples_max_cut_and_tsp.html.

[15] "NVIDIA Sets World Record for Quantum Computing Simulation With cuQuantum Running on DGX SuperPOD", Nvidia Blogs, November 9, 2021. https://blogs.nvidia.com/blog/2021/11/09/cuquantum-world-record/

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:05:33 UTC from IEEE Xplore. Restrictions apply.